Path Planning with RRT* Algorithm for Skid-Steer Mobile Robot

Diploma in Robotics - Planning and Perception Module

Brayan Gerson Duran Toconas

December, 2024

Contents

1. Introduction

This report presents the implementation of a path planning system using the **RRT* (Rapidly-exploring Random Tree Star)** algorithm for autonomous navigation of a skid-steer mobile robot. The system generates collision-free waypoints that are followed by an existing Fuzzy+PID+Lag Compensator controller.

The implementation was developed in MATLAB and integrates seamlessly with the robot control system developed in the Control Laboratory module.
Links is here:
(GitHub: Control Final Project
GitHub: Perception Final Project - Actual Report).

2. Path Planning Method: RRT*

2.1. Algorithm Selection

The **RRT* (Rapidly-exploring Random Tree Star)** algorithm was selected for this implementation. RRT* is an extension of the basic RRT algorithm that provides *asymptotic optimality*, meaning it converges to the optimal path as the number of iterations increases.

2.2. Reasons for Selection

The following reasons justify the selection of RRT* over other path planning algorithms:

1. **Asymptotic Optimality**: Unlike basic RRT, which finds *any* feasible path, RRT* continuously optimizes the path through its rewiring mechanism, converging to the optimal solution.

2. **Complex Environment Handling**: RRT* efficiently handles environments with multiple obstacles of various shapes (circles, rectangles, mazes) without requiring a grid discretization.

3. **Rewiring Mechanism**: The algorithm continuously improves the tree structure by rewiring nodes when a shorter path is found, resulting in smoother and more efficient paths.

4. **Probabilistic Completeness**: If a path exists, RRT* will find it given sufficient iterations.

5. **Anytime Algorithm**: The algorithm can be stopped at any time and still provide a valid (though possibly suboptimal) path.

2.3. Comparison with Other Algorithms

Table 1: Comparison of Path Planning Algorithms

Algorithm	Optimality	Continuous Space	Complex Obstacles
Dijkstra / A*	Optimal (grid)	No	Limited
Potential Fields	Not guaranteed	Yes	Poor (local minima)
RRT (basic)	Not optimal	Yes	Good
RRT*	**Asymptotically Optimal**	**Yes**	**Excellent**

3. Algorithm Description

3.1. RRT* Pseudocode

Algorithm 1 RRT* Algorithm

Initialize tree T with start node x_{start}
for $i = 1$ to N_{max} **do**
  $x_{rand} \leftarrow$ SampleRandom() ▷ With goal bias
  $x_{nearest} \leftarrow$ NearestNode(T, x_{rand})
  $x_{new} \leftarrow$ Steer($x_{nearest}$, x_{rand}, η)
  if CollisionFree($x_{nearest}$, x_{new}) **then**
    $X_{near} \leftarrow$ NearNodes(T, x_{new}, r)
    $x_{min} \leftarrow$ ChooseParent(X_{near}, x_{new}) ▷ Minimum cost
    AddNode(T, x_{new}, x_{min})
    Rewire(T, X_{near}, x_{new}) ▷ Optimize tree
    if Distance(x_{new}, x_{goal}) $< \epsilon$ **then**
      MarkGoalReached()
    end if
  end if
end for
return ExtractPath(T, x_{goal})

3.2. Key Components

3.2.1. Random Sampling with Goal Bias

The algorithm samples random points in the configuration space. A *goal bias* parameter (default 10%) causes the algorithm to occasionally sample the goal position directly, accelerating convergence.

3.2.2. Nearest Neighbor and Steering

For each random sample, the nearest node in the tree is found using Euclidean distance. The tree is extended from this node toward the sample by a maximum step size η.

3.2.3. Collision Checking

Before adding a new node, the algorithm verifies that the path segment is collision-free. This includes checking against:

- Circular obstacles (analytical distance check)
- Rectangular obstacles (discretized segment check)
- Robot radius safety margin

3.2.4. Parent Selection and Rewiring

The key improvement of RRT* over RRT is the parent selection and rewiring:

1. Find all nodes within radius r of the new node
2. Choose the parent that minimizes the total cost from start

3. Rewire neighboring nodes if connecting through the new node reduces their cost

3.3. Path Smoothing

After finding a path, a smoothing algorithm is applied:

1. **Shortcut Smoothing**: Randomly select pairs of non-adjacent waypoints and connect them directly if collision-free
2. **Waypoint Spacing**: Filter waypoints to maintain a minimum spacing distance, reducing the total number of waypoints

4. Implementation and Results

4.1. Project Structure

The implementation is organized in the following directory structure:

Listing 1: Project Structure

```
Perception_and_planning_lab/final_work/
|-- config/
|   |-- environment_map.m       % Map and obstacle configuration
|   |-- planner_parameters.m    % RRT* parameters
|-- planners/
|   |-- rrt_star.m              % Core RRT* algorithm
|   |-- generate_waypoints.m    % Waypoint generator wrapper
|-- utils/
|   |-- collision_check.m       % Collision detection
|   |-- path_smoothing.m        % Path optimization
|   |-- visualization_utils.m   % Plotting functions
|-- simulation/
|   |-- rrt_star_navigation_simulation.m  % Main simulation
|   |-- demo_rrt_star_planner.m           % RRT* demo
|-- tests/
|   |-- test_rrt_star_algorithm.m   % Unit tests
|   |-- test_full_navigation.m      % Integration tests
```

4.2. Configuration Parameters

The main parameters that control the algorithm behavior are shown in Table 2.

Table 2: RRT* Configuration Parameters

Parameter	Value	Description
max_iterations	4000	Maximum tree expansion iterations
step_size	0.8 m	Maximum extension distance
goal_bias	10%	Probability of sampling goal
goal_tolerance	0.5 m	Distance to consider goal reached
min_waypoint_spacing	0.3 m	Minimum distance between waypoints

4.3. Available Scenarios

Multiple test scenarios are available:

- `empty`: Empty environment for basic testing
- `simple`: 4 obstacles (circles and rectangles)
- `maze`: Simple maze with walls
- `labyrinth`: Complete labyrinth with multiple routes
- `narrow`: Narrow passages
- `extreme`: 21 obstacles, corner-to-corner navigation

4.4. Visualization Results

The visualization results are shown in Figures 1, 2, 3, and 4. And the videos of the navigation are shown in LINK.

5. How to Execute the Code

5.1. Prerequisites

- MATLAB R2020a or later
- Fuzzy Logic Toolbox (for the robot controller)
- The complete `RoboticsLab_DRM` repository: LINK

5.2. Execution Steps

5.2.1. Step 1: Navigate to Simulation Directory

Listing 2: Navigate to simulation folder

```
% In MATLAB Command Window:
cd Perception_and_planning_lab/final_work/simulation
```

5.2.2. Step 2: Run the Simulation

Listing 3: Execute main simulation

```
% Run with default scenario (labyrinth):
rrt_star_navigation_simulation

% Or specify a scenario:
rrt_star_navigation_simulation('simple')
rrt_star_navigation_simulation('maze')
rrt_star_navigation_simulation('labyrinth')
rrt_star_navigation_simulation('extreme')
```

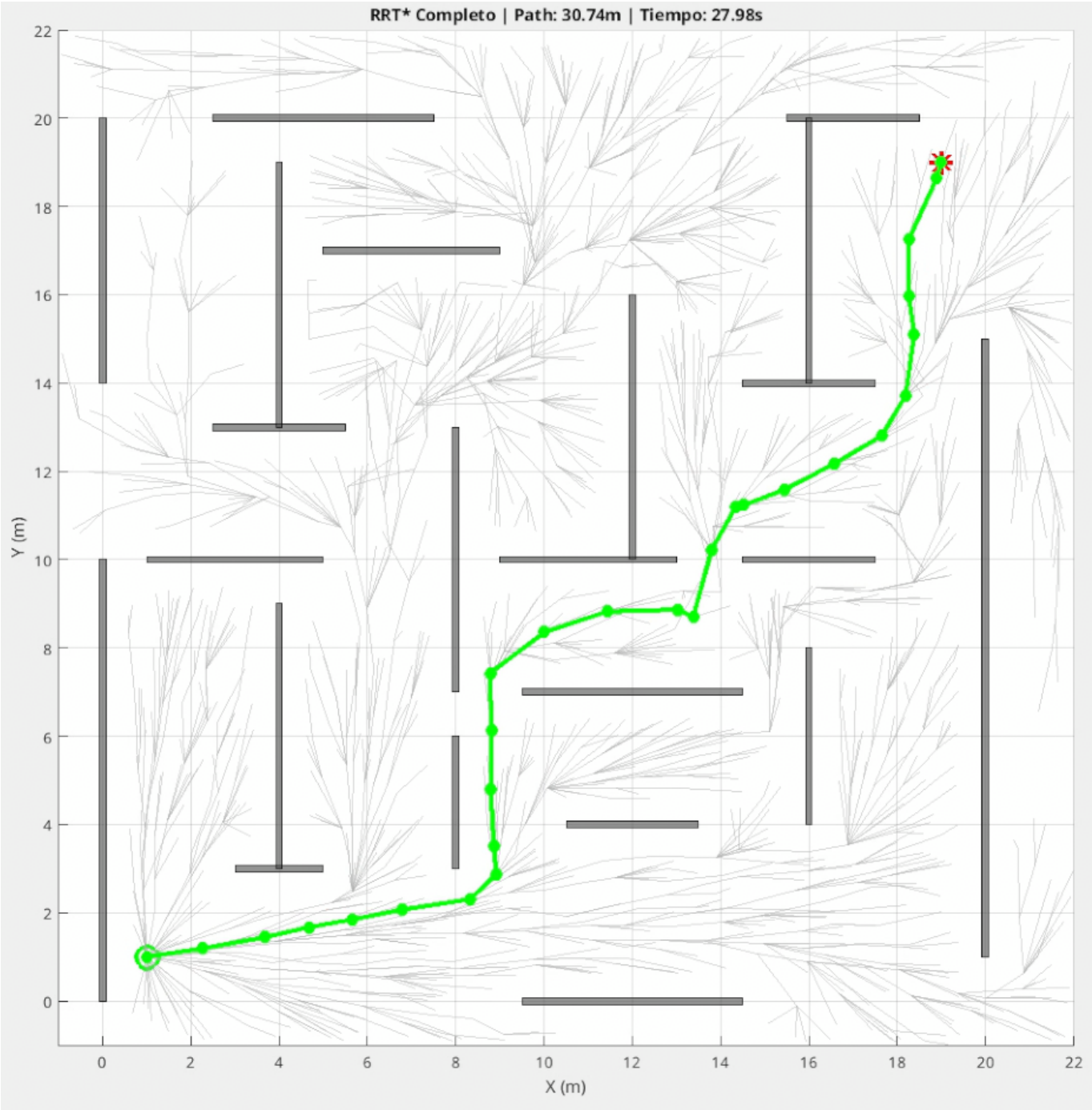

Figure 1: RRT* tree expansion showing explored nodes and optimal path found.

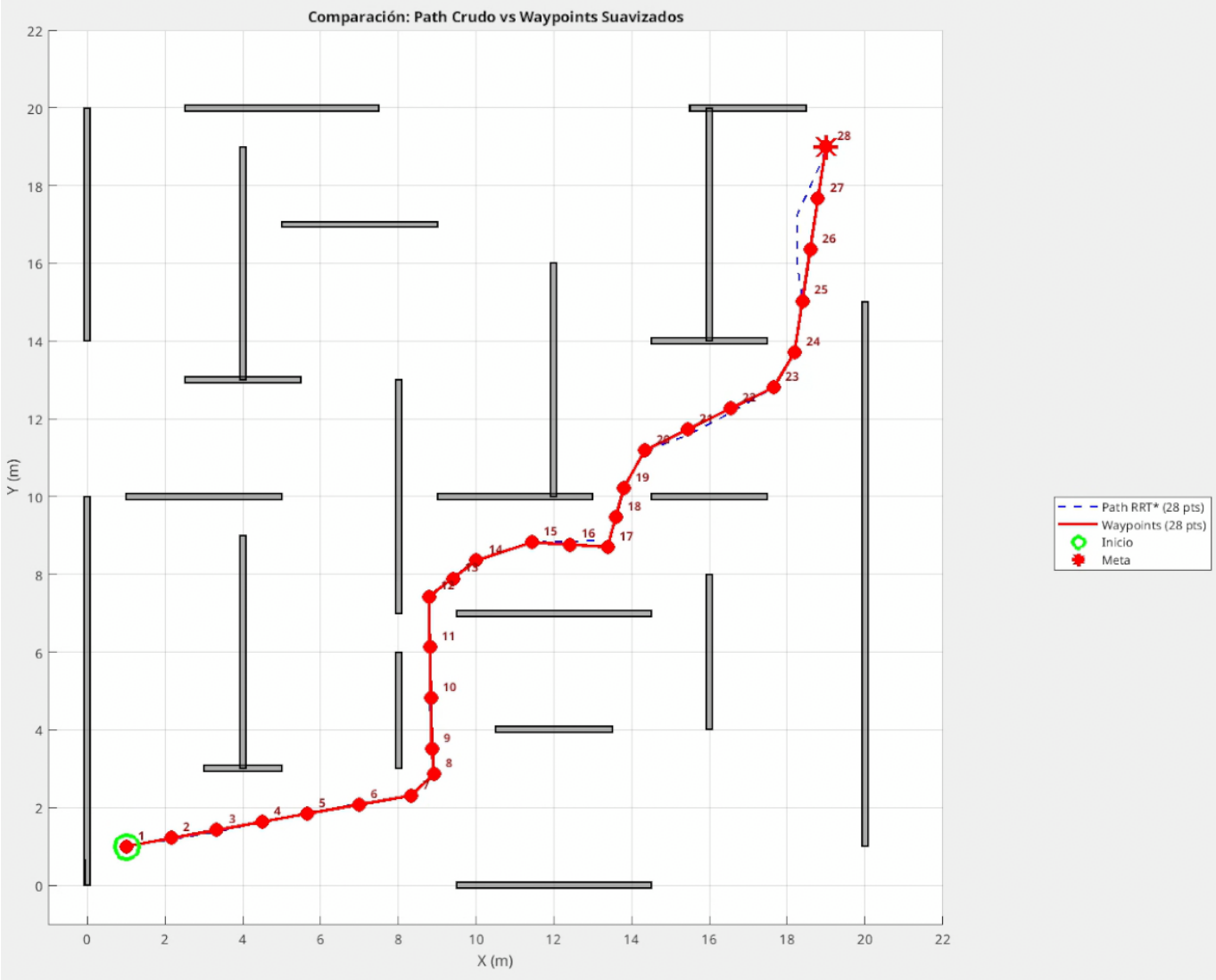

Figure 2: Comparison between raw RRT* path and smoothed waypoints.

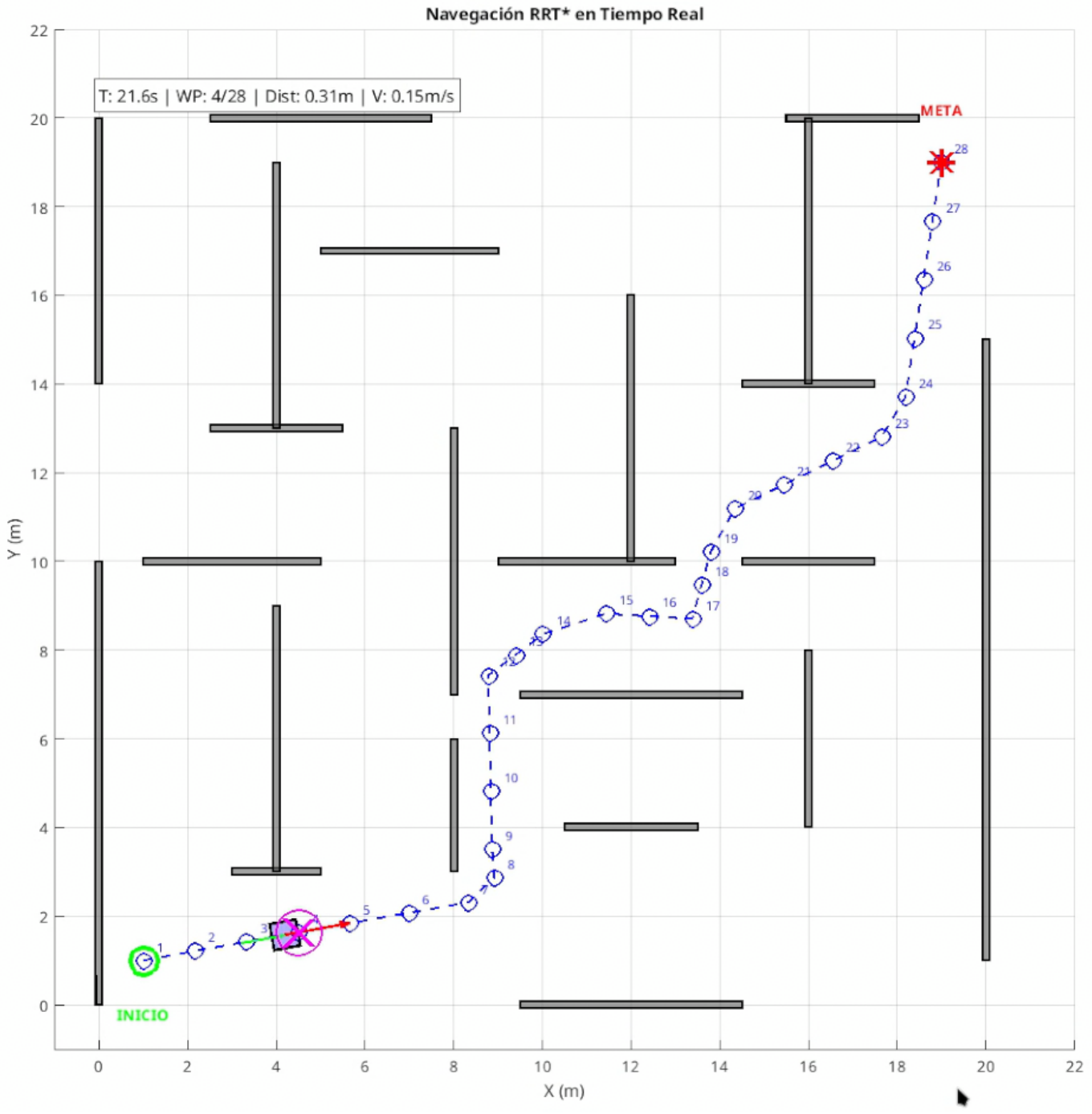

Figure 3: Real-time navigation simulation showing robot following RRT* generated waypoints.

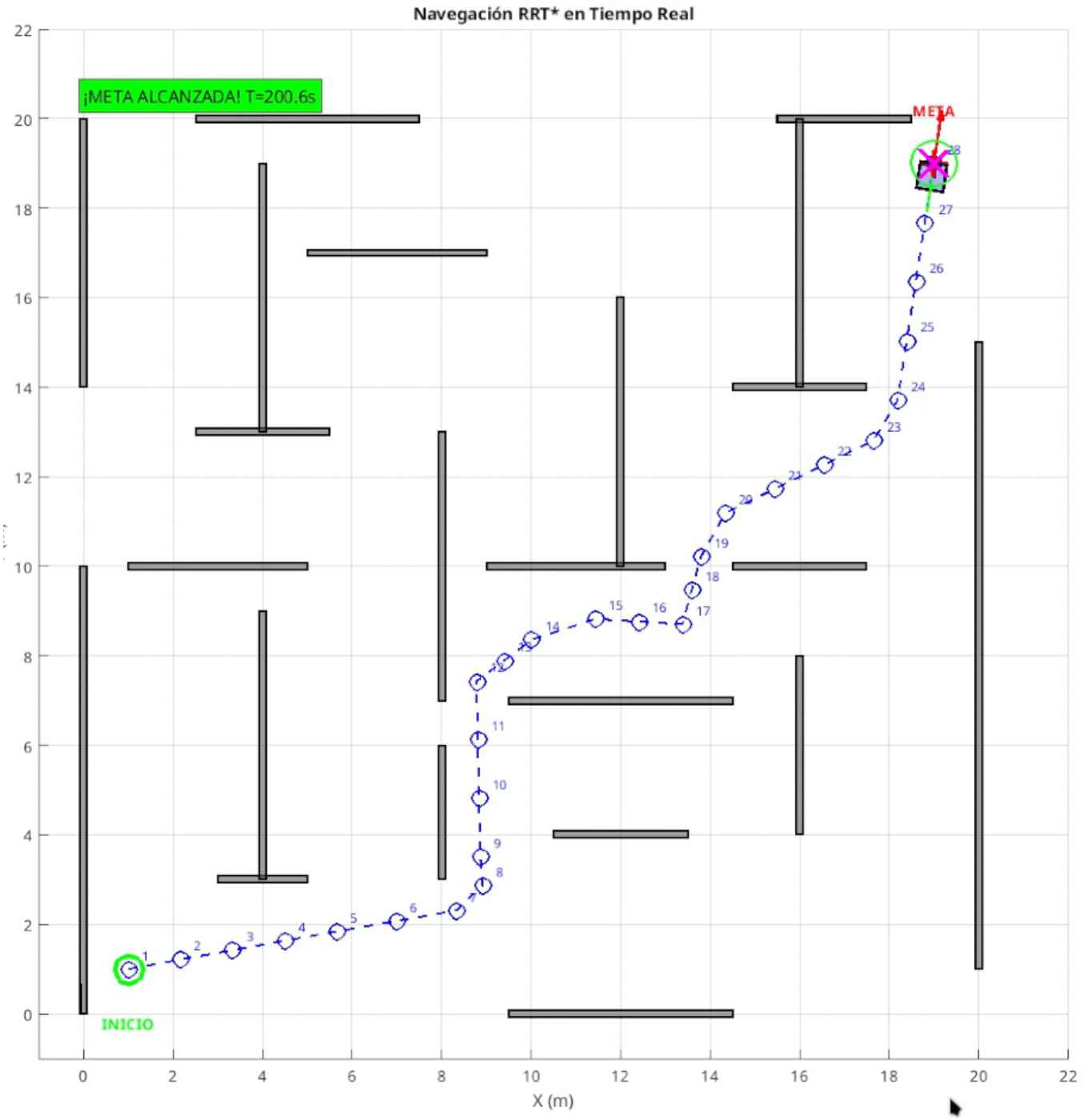

Figure 4: Path planning result in the labyrinth scenario with multiple possible routes.

5.2.3. Step 3: Observe the Results

The simulation will:

1. Generate the RRT* path with real-time tree visualization
2. Display the smoothed waypoints
3. Prompt to press any key to start navigation
4. Show real-time robot navigation following the waypoints
5. Display navigation statistics upon completion

5.2.4. Alternative: RRT* Demo Only

To visualize only the path planning algorithm without robot navigation:

Listing 4: Run RRT* demo

```
cd Perception_and_planning_lab/final_work/simulation
demo_rrt_star_planner
```

5.2.5. Running Tests

Listing 5: Execute tests

```
cd Perception_and_planning_lab/final_work/tests

% Unit tests for RRT* algorithm:
test_rrt_star_algorithm

% Integration test with robot controller:
test_full_navigation
```

6. Conclusion

The RRT* algorithm was successfully implemented and integrated with the existing skid-steer robot controller. The implementation demonstrates:

- Efficient path planning in complex environments with multiple obstacles
- Asymptotically optimal paths through the rewiring mechanism
- Real-time visualization of tree expansion and path optimization
- Seamless integration with the Fuzzy+PID+Lag Compensator controller
- Configurable parameters for different scenarios and requirements

The modular architecture allows easy extension to include dynamic obstacles, multi-robot coordination, or alternative planning algorithms in future work.

A. Core Algorithm Code

Listing 6: RRT* Main Function (rrt_star.m) - Excerpt

```
function [path, tree, stats] = rrt_star(start, goal, map, params)
% RRT* Path Planning Algorithm
% Finds asymptotically optimal path from start to goal

%% Initialize tree
tree.nodes = start';      % [x, y] for each node
tree.parent = 0;          % Parent index (0 = root)
tree.cost = 0;            % Cost from start

goal_found = false;
goal_idx = -1;

%% Main loop
for iter = 1:params.max_iterations
    % Sample random point (with goal bias)
    if rand() < params.goal_bias
        x_rand = goal;
    else
        x_rand = [map.x_min + rand()*(map.x_max - map.x_min);
                  map.y_min + rand()*(map.y_max - map.y_min)];
    end

    % Find nearest node
    [~, nearest_idx] = min(vecnorm(tree.nodes - x_rand', 2, 2));
    x_nearest = tree.nodes(nearest_idx, :)';

    % Steer toward random point
    direction = x_rand - x_nearest;
    dist = norm(direction);
    if dist > params.step_size
        direction = direction / dist * params.step_size;
    end
    x_new = x_nearest + direction;

    % Check collision
    if ~collision_check(x_nearest, x_new, map.obstacles, map.robot_radius)
        % Find neighbors within radius
        neighbor_radius = min(params.neighbor_radius_factor * ...
            sqrt(log(size(tree.nodes,1)+1) / (size(tree.nodes,1)+1)), ...
            params.step_size * 3);

        dists = vecnorm(tree.nodes - x_new', 2, 2);
        neighbor_idxs = find(dists < neighbor_radius);

        % Choose best parent (minimum cost)
        min_cost = tree.cost(nearest_idx) + norm(x_new - x_nearest);
        best_parent = nearest_idx;

        for n = neighbor_idxs'
            x_near = tree.nodes(n, :)';
            cost_through_n = tree.cost(n) + norm(x_new - x_near);
            if cost_through_n < min_cost
                if ~collision_check(x_near, x_new, map.obstacles,
                   map.robot_radius)
                    min_cost = cost_through_n;
                    best_parent = n;
                end
```

```
            end
        end

        % Add new node
        tree.nodes = [tree.nodes; x_new'];
        tree.parent = [tree.parent; best_parent];
        tree.cost = [tree.cost; min_cost];
        new_idx = size(tree.nodes, 1);

        % Rewire neighbors
        for n = neighbor_idxs'
            x_near = tree.nodes(n, :)';
            cost_through_new = min_cost + norm(x_near - x_new);
            if cost_through_new < tree.cost(n)
                if ~collision_check(x_new, x_near, map.obstacles,
                    map.robot_radius)
                    tree.parent(n) = new_idx;
                    tree.cost(n) = cost_through_new;
                end
            end
        end

        % Check if goal reached
        if norm(x_new - goal) < params.goal_tolerance
            goal_found = true;
            goal_idx = new_idx;
        end
    end
end

% Extract path by backtracking
if goal_found
    path = [];
    idx = goal_idx;
    while idx > 0
        path = [tree.nodes(idx, :); path];
        idx = tree.parent(idx);
    end
else
    path = [];
end
end
```